Assignment 2

COMP 250 Fall 2020

posted: Tuesday, Oct. 13, 2020
due: Monday, Nov. 2, 2020 at 23:59

Learning Objectives

General Instructions

- **Submission instructions**
  - Late assignments will be accepted up to 2 days late and will be penalized by 10 points per day. Note that submitting one minute late is the same as submitting 23 hours late. We will deduct 10 points for any student who has to resubmit after the due date (i.e. late) irrespective of the reason, be it wrong file submitted, wrong file format was submitted or any other reason. This policy will hold regardless of whether or not the student can provide proof that the assignment was indeed "done" on time.
  - Don't worry if you realize that you made a mistake after you submitted : you can submit multiple times but only the latest submission will be kept. We encourage you to submit a first version a few days before the deadline (computer crashes do happen and codePost may be overloaded during rush hours).
  - These are the files you should be submitting on codePost:
    - `Polynomial.java`
    - `SLinkedList.java`

    Do not submit any other files, especially .class files. Any deviation from these requirements may lead to lost marks
- Please note that the classes you submit should be part of a package called `assignment2`.
- **Do not change any of the starter code that is given to you. Add code only where instructed, namely in the "ADD CODE HERE" block.** You may add helper methods to the `Polynomial` class, but you are not allowed to modify any other class except for the method you need to implement in the `SLinkedList` class.
- The assignment shall be graded automatically. Requests to evaluate the assignment manually shall not be entertained, so please make sure that you follow the instruction closely or your code may fail to pass the automatic tests. Note that for this assignment, you are NOT allowed to import any other class. **Any failure to comply with these rules will give you an automatic 0.**

- Whenever you submit your files to codepost, you will see the results of some exposed tests. These tests are a mini version of the tests we will be using to grade your work. If your code fails those tests, it means that there is a mistake somewhere. Even if your code passes those tests, it may still contain some errors. We will test your code on a much more challenging set of examples. We highly encourage you to test your code thoroughly before submitting your final version.
- The starter code includes a `Minitester` class that you can run to test if your methods are correct. This class is equivalent to the exposed tests on codePost. Please note that these tests are only a subset of what we will be running on your submissions. We encourage you modify and expand these classes. You are welcome to share your tester code with other students on Piazza. Try to identify tricky cases. Do **not** hand in your tester code. Note that your code will be tested **on valid inputs only**.
- You will automatically get 0 if your code does not compile.
- Failure to comply with any of these rules will be penalized. If anything is unclear, it is up to you to clarify it by asking either directly a TA during office hours, or on the discussion board on Piazza.

Learning Objectives

There are several learning goals for this assignment.

First, you will understand much better how to manipulate polynomials. Polynomials are heavily used in computer science and in mathematics and also in many other sciences. They are the basis of many models in physics and chemistry and other natural sciences, as well as in the social sciences such as economics. For example, they are commonly used to approximate functions. An example is a Taylor series expansion of a smooth function. Another example is that any continuous function on a closed interval of the real line can be approximated as closely as one wishes using a polynomial.

Second, in this assignment you will also get some experience working with linked lists. You will implement part of the linked list class and you will use this data structure to represent large polynomials.

Third, in this assignment we will start to focus also on the efficiency of your algorithms. You will learn to look at code with a more critical eye, without only focusing on the correcness of your methods.

Lastly, this assignment will also give you more practice programming in Java! Although COMP 250 is not a course about how to program, programming *is* a core part of computer science and the more practice you get, the better you will become at it.

Introduction

In mathematics, a polynomial is an expression constructed from variables and constants, using the operations of addition, subtraction, multiplication, and constant non-negative integer exponents. For example, $5x^3 4x + 7$ is a polynomial. In general, a polynomial can be written as

$$p = \sum_{i=0}^{m} cx^i = c_0 + c_1 x + c_2 x^2 + \cdots + c_m x^m$$

where the coefficients c_i are real numbers and $c_m \neq 0$. As explained below, in this assignment, the coefficients c_i and the variables x will be integers, represented with the Java BigInteger class.

Instructions and Starter Code

We will use a singly linked list data structure to represent polynomials and perform basic operations on them, such as addition and multiplication. Essentially, you will write a Polynomial class that builds upon the functionality of a linked list class. The starter code defines four classes which are as follows:

- `Polynomial` - This class defines a polynomial in one variable with positive integer exponents. Most of your work goes into this file. You should use the methods provided with the linked list class to implement the methods of this class. You will notice that the template we provided

use Java `BigInteger` for storing the polynomial coefficient and variable. We intentionally chose `BigInteger` over floating point (e.g. double) to avoid numerical issues associated with polynomial evaluation.

- `SLinkedList` – This class implements a generic singly linked list. You do not have to implement the functionality of linked list as it is already provided with the starter code. However, you must implement the cloning functionality (as explained see below) for performing a deep copy of the list.
- `Term` – This is a simple class that represents a single term of a polynomial. Example: $5x^2$. This class is also provided with the starter code. You do not have to modify this class, but you are required to understand what it does and how.
- `DeepClone` – This is a small user defined interface for enforcing deep copy functionality. You are not required to understand Java interfaces for completing this assignment as the underlying complexity is handled by the starter code. We will discuss Java interfaces in the lectures a few weeks from now.

Valid polynomial representation

Any method that outputs a polynomial or modifies an existing polynomial (adding a term or multiplying by a term) must ensure that this polynomial satisfies the following:

- There must be no term in the polynomial with zero coefficient. e.g: term $0x^3$ is **not** allowed.
- The exponents in the list must be in **decreasing** order
- Exponents can be non-negative integers only e.g. a term x^{-2} is **not** allowed.
- There can be at most one term for each exponent.
- A polynomial can be zero, i.e. $p(x) = 0$. In this case, the polynomial must be an empty linked list with no term. i.e. size of the linked list should be zero.

Methods you need to implement [100 points total]

The methods are listed below. See the starter code for method signatures and more details about what the methods do.

Your implementations must be efficient. For each method below, we indicate the worst case run time using $O()$ notation. This worst case may depend on either the order m of the polynomial as in the definition above, or it may depend on the number n of terms in the polynomial. Note that $n \leq m + 1$.

- `SLinkedList.deepClone()` [15 points]
  Returns a deep copy of the linked list object. It should step through the linked list and make a copy of each term. The runtime of this method should be $O(n)$.

- `Polynomial.addTerm()` [30 points]
  Add a new term to the polynomial. Use the methods provided by the `SLinkedList` class. The runtime of this method should be $O(n)$.

- `Polynomial.add()` [10 points]
  This is a `static` method that adds two polynomials and returns a new polynomial as result. You may use any of the class methods. Be careful not to modify either of the two polynomials. The runtime of this method should be $O(n_1 + n_2)$ where n_1, n_2 are the number of terms in the two polynomials being added.

- `Polynomial.multiplyTerm()` [15 points]
  The method multiplies each of the polynomial's terms by an argument term. The runtime of this method should be $O(n)$.

- `Polynomial.multiply()` [10 points]
  This is a `static` method that multiplies two polynomials and returns a new polynomi al as result. Careful not to modify either of the two input polynomials. The runtime of this method should be $O(n_1 n_2)$ where n_1, n_2 are the number of terms in the two polynomials being multiplied.

  Hint: Please note that using `Polynomial.multiplyTerm()` and `Polynomial.add()` you can have an implementation of this method that runs in $O(n_1 \cdot n_2 \cdot n_2)$. To find a more efficient implementation, think about using an additional data structure.

- `Polynomial.eval()` [20 points]
  This method evaluates the polynomial given a value for x using Horner's method. The variable x is of `BigInteger` data type, as mentioned earlier. Horner's method greatly speeds up the evaluation for polynomials with many terms. It does so by not having to re-compute the x^i fresh for each term. Note that you **should not** use the `Term.eval()` method to evaluate a particular term. That method is provided for you only to help with testing, namely to ensure that your implementation of Horner's method is correct.

  A good resource for understanding Horner's method is:
  https://www.geeksforgeeks.org/horners-method-polynomial-evaluation/

  Alternatively, you may use any other resource explaining Horner's method. Note the method described in the above resource works only when a polynomial has all $m + 1$ terms (up to order m). However, your polynomial representation may not have all terms. Hence, you must modify your implementation accordingly.

  We emphasize that the efficiency of your solution is very important. If a polynomial of order m has all $m+1$ terms, then normal brute force polynomial evaluation would take time $O(m^2)$, that is, if you were to evaluate each term individually and sum the results. Horner's algorithm is more efficient, namely the evaluation of the polynomial would take time $O(m)$. As such, we will evaluate you on polynomials with a very large number of terms that will "stress test" the $O()$ efficiency of your `Polynomial.eval` method, as well as its correctness. Your algorithm should take time $O(m)$.

  One heads up about terminology: when we discuss the complexity of various operations in

this assignment, e.g. on Piazza, we will use term "order" in two different ways. One way is to refer to the order m the polynomial. Another is to refer to the $O()$ complexity, where in general people often say "order" for "big O", that is, people often refer to $O(1)$, $O(n)$, $O(n^2)$ as "order 1, n, and n squared" instead of "big O of 1, big O of n and big O of n squared", respectively.

Points breakdown

Here is a table with the breakdown of the points for all the methods you have to implement in this project:

Method name	Correctness	Time complexity	Total
SLinkedList.deepClone()	10	5	15
Polynomial.addTerm()	20	10	30
Polynomial.add()	7	3	10
Polynomial.multiplyTerm()	10	5	15
Polynomial.multiply()	5	5	10
Polynomial.eval()	10	10	20
Total	62	38	100

Note that the tests provided test correctness and efficiency of `Polynomial.multiplyTerm()` and of `Polynomial.eval()`, while they partially test for correctness of `Polynomial.addTerm()`. For all the other methods it is up to you to test and debug your code.